

MSSE Capstone Project

Sprint 0 Review, Retrospective & Sprint 1 Alignment

Meeting Minutes

TrustAI Marketplace

Date	July 15, 2026
Time	Approximately 7:07 PM CAT / 10:07 AM PDT
Duration	Approximately 40 minutes 45 seconds
Location	Virtual meeting (Google Meet)
Meeting Type	Sprint 0 review and retrospective, architecture review, repository alignment, and Sprint 1 planning
Prepared For	TrustAI Marketplace MSSE Capstone team records

1. Attendees and Meeting Roles

Five active team members attended and participated in the Sprint 0 review and Sprint 1 alignment discussion:

- Ahmed Al-Mandalawi - Product Owner and AI Analysis Lead
- Ranga Nyamadzawo - Backend and platform contributor; presenter of the architecture and boilerplate
- Mulima Chibuye - Scrum Master and Project Manager
- Adrian Muchatibaya - Frontend lead and wireframe owner
- Samar El Ghandour - QA/DevOps lead and backend contributor

2. Purpose of the Meeting

This meeting was intended to close out Sprint 0 and get the team properly lined up for Sprint 1. The main focus was to review the work completed so far, make sure the proposed architecture still matched the project vision, agree on how the team would work in GitHub, and remove any remaining access or ownership issues before development picked up pace.

- Review the end-to-end architecture and bounded-context diagrams.
- Confirm the agreed technology stack and the role of the external LLM.
- Clarify the boundaries between identity and access, listing management, and risk analysis.
- Discuss how risk should be presented to users and whether a percentage score would be defensible.
- Agree on a common design direction for the frontend.
- Clarify how Trello, GitHub, meeting notes, and repository documentation would be used together.
- Confirm areas of ownership and how team members would support one another across workstreams.
- Review what was complete from Sprint 0, what was still open, and what needed to move into Sprint 1.
- Agree on branches, pull requests, reviews, branch protection, and testing expectations.

- Settle which repository the team would use for active development and confirm access for every member.
- Agree on the purpose and naming of the weekly meetings going forward.

3. Sprint 0 Review

The team acknowledged that Sprint 0 was ending with several setup items still unfinished. Even so, enough groundwork had been completed for development to begin, and everyone agreed that the project could not afford to wait for every setup task to be perfect before starting Sprint 1.

3.1 Work completed or substantially completed

- An end-to-end architecture diagram and a bounded-context diagram had been drafted and shared.
- A runnable boilerplate had been created with route signatures, Pydantic schemas, workstream stubs, story-related TODO markers, skipped tests, CI configuration, Docker/environment conventions, and a working `/api/health` endpoint.
- The Trello board had been populated and Sprint 1 items had been moved into the Next-Up area.
- Adrian had started the wireframes, and that work was treated as in progress.
- The repository included a documentation structure and records of earlier project decisions.
- An initial GitHub Actions/CI skeleton was already present.

3.2 Work still open at the end of Sprint 0

- The team had not yet chosen or documented a final design system and visual template.
- The architecture and bounded-context diagrams still needed a proper team review rather than only a quick verbal reaction.
- The team still needed to confirm whether the MVP would remain one FastAPI deployable with clear internal modules or become multiple independently deployed services.
- Repository administration, source-of-truth, and deployment responsibilities still needed to be written down clearly.
- Branch protection, required checks, reviewer rules, and naming conventions were not yet configured or documented.
- The LLM strategy and final acceptance criteria for risk labels were still open.
- The complete Dockerized application and VPS deployment had not yet been finished.
- Not every team member had yet tested the full clone, branch, pull-request, review, and merge workflow.
- Sprint 1 ownership needed to be visible and up to date for every member.

Sprint timing: Sprint 0 was treated as closed on July 15. Sprint 1 would begin immediately on July 16, while the remaining setup tasks continued alongside feature work.

4. Architecture Review

4.1 End-to-end view

Ranga opened the meeting by sharing the end-to-end architecture and the bounded-context diagram. He asked whether the proposed structure still reflected what the team had discussed previously and whether anyone wanted components moved, removed, or revised. The first reactions were generally positive, although several people had only reviewed the diagrams briefly and did not offer detailed feedback at that point.

The proposed system consisted of a React frontend, a FastAPI/Python backend, PostgreSQL for persistence, an external LLM integration, and GitHub Actions for testing, building, and deployment work.

The team's attention was mainly on whether the responsibilities were divided sensibly and whether the structure would allow people to work without blocking one another.

4.2 Bounded contexts and module boundaries

Ranga explained the design in terms of three main capability areas inside the TrustAI application, with the LLM treated as an external dependency:

- Identity and access management: users, credentials, authentication tokens, login/session handling, and secure access to the application.
- Listing management: the listing text, URL, or structured information submitted by the user; storage of that submission; and the user's previous submissions and analysis history.
- Risk analysis: scam indicators, the risk level, the final verdict or recommendation, prompt/provider interaction, and interpretation of the model response.
- External LLM: a model provider outside the TrustAI domain. The team did not plan to train or customize its own model for the MVP, so the application would connect to a suitable provider through an interface.
- Shared contracts: the Pydantic schemas and API contracts that define what each module receives and returns.

No major concerns were raised about these boundaries. The general view was that they matched the direction agreed in earlier meetings and gave the team a reasonable starting point for Sprint 1. Everyone also understood that the diagrams were still drafts and could be adjusted as the implementation became clearer.

4.3 How the team interpreted “microservices”

Mulima noted that each bounded context could eventually become its own service or container, with its own structure and possibly its own data ownership. The main benefit discussed was the ability to work in parallel: once the inputs and outputs are agreed, one person can build a module without waiting for another person to finish theirs.

At the same time, the current boilerplate was based on a shared FastAPI foundation. The team therefore agreed to preserve clear capability boundaries and explicit contracts while leaving the final deployment shape open for a later architecture decision. The important requirement was that the diagrams, written architecture and implemented code eventually tell the same story.

5. Risk Analysis and LLM Output

The longest product discussion concerned how TrustAI should communicate risk to the user. An earlier version of the idea had included a percentage score. Ranga raised a valid concern that a percentage could look more scientific or precise than the team could honestly support. The project did not have a ground-truth dataset or a calibrated scoring model that could prove that a listing was, for example, “82% risky.”

The alternative discussed was to use clear categories, possibly supported by color, such as Low, High, and Critical. The user would also see the specific indicators and the reasoning behind the result. This would make the output easier to explain and defend, and it would present TrustAI as a decision-support tool rather than as a system claiming certainty it did not have.

Agreed direction: TrustAI would not present an unsupported probability percentage unless the team later implemented and validated a reproducible scoring method. The preferred Sprint 1 direction was an explainable categorical risk result supported by visible indicators, plain-language reasoning, caveats and a recommendation.

The exact labels, colors, and wording were not finalized during this meeting. Ahmed, as Product Owner and AI Analysis Lead, was expected to define the final user-facing contract and make sure it could be implemented consistently in the backend, frontend, and tests.

The team also agreed that the LLM should remain behind a provider or adapter interface. Its raw response should be validated against the agreed schema and translated into TrustAI's own internal format. This avoids tying the application directly to one provider and makes it easier to use deterministic mock responses during development and CI.

6. API Contracts and Parallel Development

The shared kernel was discussed as the set of agreed contracts that allows the different parts of the system to work together. In practical terms, this means defining the request and response schemas before the team gets too far into feature development.

- Authentication, listings, and analysis should have clear request and response models.
- The shared area should contain only stable items such as identifiers, enums, validation models, common errors, and other cross-module contracts.
- The owner of each area should document what that area expects from the others.
- The existing schemas and route signatures in the boilerplate should be reviewed rather than accepted automatically.
- Skipped tests and deterministic mocks can be used as executable expectations. A story is not complete until the relevant tests are enabled and passing.
- Each README should explain what its part of the codebase is responsible for and how it connects to the rest of the system.

The group liked this approach because it allows people to build against an agreed promise instead of waiting for another module to be fully implemented.

7. Technology Decisions

7.1 Frontend

React was confirmed as the frontend framework. The team considered it familiar enough for the project and suitable for the size of the MVP. Frontend work should therefore continue within the existing React structure rather than introducing another framework.

7.2 Backend

FastAPI and Python were confirmed for the backend. This applies to authentication, listings, analysis, and any other backend capability unless the team formally records a different decision later.

7.3 Database and containers

PostgreSQL remained the primary database. Docker would be used to give the team a consistent development environment and to support the eventual Docker Compose deployment on the selected VPS. The group agreed that deployment work should continue in parallel, but not at the expense of the core Sprint 1 functionality.

7.4 CI/CD and health check

The boilerplate already contained an initial CI workflow, Docker and environment conventions, skipped tests, and a working `/api/health` endpoint. These were seen as a useful starting point, not as finished DevOps work. The pipeline will need to grow as real features and tests replace the placeholders.

8. Frontend Design and Consistency

The team returned to the need for a common look and feel. The concern was straightforward: even if different people build different pages, the finished product should still look and behave like one application. The frontend therefore needs a shared color palette, typography, reusable components, spacing rules, and consistent handling of loading, validation, and error states.

Adrian's wireframes were acknowledged as the current starting point. Adrian will lead the design-system recommendation and frontend implementation, with support from Mulima. Other team members can comment or help, but Adrian is responsible for keeping this work moving and bringing a proposal back to the group.

- Choose or define a common visual system before several pages are built independently.
- Cover registration/login, listing submission, results, history, loading, validation, and error states.
- Make sure the risk presentation is understandable and accessible; color should not be the only signal.
- Document reusable components and frontend conventions in the repository.
- Use pull requests so the relevant reviewers can check consistency before changes are merged.

9. Project Tracking and Documentation

A question was raised about how the team would keep track of ownership and produce a coherent project record, rather than simply picking isolated tasks. The answer was that Trello, GitHub, meeting notes, and the repository documentation would be used together.

9.1 Trello

- Trello remains the working board for backlog priority, ownership, and status.
- Each person should assign a card to themselves before starting work.
- Critical Sprint 1 items should be completed before lower-priority work.
- People may pick up additional work when they have capacity, as long as the sprint goal remains protected.
- Statuses should be kept current so the team can see what is active, blocked, in review, or complete.

9.2 GitHub

- GitHub will hold the code, branches, pull requests, reviews, CI results, and implementation history.
- Work should be completed on task or feature branches rather than directly on main.
- Every pull request should be assigned to an appropriate reviewer.
- Repository READMEs and the /docs area should be updated through pull requests just like code.
- Documentation should be treated as part of the deliverable, not something to reconstruct at the end.

9.3 Meeting notes and project history

10. Ownership and Team Strengths

The team clarified that ownership means accountability, not isolation. Each workstream needs someone who will drive it, answer questions, and coordinate dependencies. That does not prevent others from helping, reviewing, pairing, or taking additional cards once their own work is under control.

Member	Agreed strength or focus
Ranga	Comfortable leading or heavily supporting backend work. His day-to-day background is DevOps/platform engineering, and he was willing to deepen his backend work for the project.
Adrian	Preferred frontend work and had already started the wireframes. He was comfortable leading the frontend while learning from backend work where needed.
Samar	Preferred backend work while continuing to cover QA and DevOps responsibilities.
Ahmed	Preferred backend work but also stated that he could contribute to the frontend. His main ownership remains product direction and AI analysis.
Mulima	Identified as full-stack and able to support Adrian on the frontend while continuing the Scrum Master and project-management work.

10.1 Working allocation agreed during the meeting

- Frontend: Adrian leads, with Mulima supporting. They will coordinate wireframes, the design system, layouts, and frontend pages.
- Backend: Ranga and Ahmed lead the main backend work, with Samar contributing to backend tasks as well as QA and DevOps.
- AI analysis: Ahmed owns the provider interaction, prompt design, analysis contract, risk and recommendation logic, buyer questions, price plausibility, and Product Owner acceptance criteria.
- Identity and access: Ranga used this as the main example of his backend ownership area.
- QA/DevOps: Samar will review the existing CI setup, branch protection, test gates, infrastructure, and deployment needs. She can also support backend and test implementation.
- Cross-functional support: Team members can help outside their main area, particularly where APIs, integration, tests, UI behavior, or deployment dependencies meet.

11. DevOps, Testing, and Quality Gates

Samar asked whether she should start with DevOps or join backend/frontend work. The team agreed that the CI skeleton already in the boilerplate did not mean DevOps was finished. She could progress the infrastructure and quality work in parallel while also helping with backend and testing tasks.

- Review and strengthen the existing CI workflow.
- Add branch protection, required reviews, and test gates before code enters main.
- Enable and pass the tests connected to each story as part of the Definition of Done.
- Add unit, integration, and later end-to-end tests as real features replace the stubs.
- Check Docker behavior across the team's different operating systems.
- Prepare and document the VPS, TLS, database backup, and deployment workflow.
- Keep DevOps focused on enabling the Sprint 1 goal; lower-priority infrastructure work should not replace critical feature delivery.

12. Branching, Pull Requests, and Reviews

The team agreed that every task or story should be completed on its own branch. The branch name should identify the work being done. Once the work is complete and tested, it should be pushed and submitted through a pull request rather than merged directly into main.

Agreed workflow: Select and assign a Trello card -> create a branch named for the task/story -> implement and test -> push the branch -> open a pull request -> assign the right reviewer -> address comments and pass the checks -> merge into main.

Samar agreed to prepare a short branch-naming and workflow guide by the following day. Ranga explained that the original boilerplate pull request had been merged without reviewers because the team had not yet been added. Everyone agreed that this should be treated as an early exception and not the normal process once permissions and branch rules were in place.

Reviewers should be chosen according to the area affected. For example, Mulima can review frontend work with Adrian, while backend, AI, and DevOps changes should go to the relevant owners. Samar will help configure the protections and checks needed to support this workflow.

13. Repository Decision and Access

13.1 Question raised

Near the end of the meeting, Ahmed asked whether the team would keep using the newer repository created by Ranga or return to the repository that Ahmed had shared earlier. By that point, some members had already cloned the newer repository and it contained the runnable boilerplate.

13.2 Decision made during the meeting

At this point in the project, the team agreed to continue active development in Ranga-Schooling/trustai-marketplace. The decision was practical: the repository was further ahead, the runnable boilerplate was already present, and team access was working. The group wanted to avoid another repository transition while Sprint 1 was beginning.

Ahmed accepted the invitation during the call and said that he would need to reorient himself to the new repository and make the required changes. Everyone was asked to clone the repository, create a branch, and test the pull-request flow so that access problems could be found immediately rather than later in the sprint.

13.3 Deployment and the existing domain

The team confirmed that Ahmed's existing domain arrangement could still be used for the project. The exact deployment source, CI trigger, repository relationship, and operational ownership still need to be documented before automated deployment is finalized.

14. Communication and Weekly Meetings

The team agreed that the regular mid-sprint meetings should be called stand-ups or weekly alignment meetings. Sprint Reviews and Retrospectives should remain tied to the end of a sprint.

- Design questions and blockers should be raised as soon as they come up rather than saved for the next call.
- The team should use the shared channels and respond as soon as practical across time zones.
- Urgent blockers may require direct messages or a quick call.

- Trello, GitHub, and discussion threads should stay current so one person's absence does not stop the rest of the team.
- The group recognized that two weeks of July had already passed and that the remaining schedule was tight.

15. Decisions Made

ID	Decision	Why	Follow-up
D-01	Use React for the frontend.	The framework was already agreed and is familiar enough for the MVP.	Continue within the React skeleton and apply the common design system.
D-02	Use FastAPI/Python for the backend.	This matches the earlier planning and the current boilerplate.	Keep backend work within the agreed structure and contracts.
D-03	Keep Identity & Access, Listing Management, and Risk Analysis as the main capability boundaries.	The boundaries separate responsibilities and support parallel work.	Review the diagrams and record whether these remain modules or become separate deployables.
D-04	Keep the external LLM outside the TrustAI domain and access it through a provider/adaptor interface.	This avoids tight coupling to one provider and supports mocks and provider switching.	Ahmed to refine the provider contract, prompts, validation, and accepted output.
D-05	Use explainable risk categories rather than an unsupported percentage.	The team does not have a validated model that justifies precise probabilities.	Finalize the labels, colors, evidence fields, caveats, and recommendation wording.
D-06	Use Pydantic schemas and API contracts as the shared agreement between workstreams.	This lets the team build in parallel against known inputs and outputs.	Review and approve the current boilerplate contracts before too much implementation is built around them.
D-07	Use Trello for backlog/status and GitHub for code, reviews, CI, and implementation history.	Together with the meeting notes, these provide the project record.	Every member must keep an assigned task and current status.
D-08	Use one common frontend design system.	The finished application should look and behave consistently.	Adrian leads, Mulima supports, and the team reviews.
D-09	Use task/story branches and pull requests.	This protects main, creates a review trail, and supports testing.	Samar to document naming and help configure protections and checks.
D-10	Continue active development in Ranga-Schooling/trustai-marketplace.	It contained the most advanced runnable boilerplate and had working team access.	All members to accept access, clone, create a task branch, and test the PR process.
D-11	Begin Sprint 1 immediately after July 15.	Sprint 0 ended with some setup work still open and the schedule was already behind.	Carry the unfinished setup work into Sprint 1 without losing focus on the critical path.
D-12	Call the regular mid-sprint meetings stand-ups or alignment meetings.	Formal reviews and retrospectives should remain linked to sprint boundaries.	Continue asynchronous discussion between meetings.

16. Action Items

Owner	Action	Target	Status
Ahmed	Review and clone the working repository; create a test branch and PR; read the READMEs and identify missing or incorrect product documentation.	Immediate / Sprint 1	Started
Ahmed	Define the analysis output and Product Owner acceptance criteria for risk labels, indicators, recommendation, buyer questions, limitations, and price plausibility.	Early Sprint 1	Open
Ahmed	Begin the AI provider/interface and risk-output work, making sure it matches the Pydantic schemas and frontend results design.	Sprint 1	Open
Ahmed	Bring forward any useful documentation and decisions from the earlier repository through reviewed pull requests and help document the domain/deployment arrangement.	Sprint 1	Open
Ranga	Maintain and explain the boilerplate; continue backend work around authentication, listings, contracts, and integration; review corrections raised by the team.	Sprint 1	In progress
Ranga	Update architecture and README material based on team comments and make sure the written deployment approach matches the actual VPS/Docker plan.	Early Sprint 1	Open
Mulima	Maintain Trello, make sure every member has an assigned card, keep priorities clear, update statuses, and track Sprint 0 work carried into Sprint 1.	Immediate / ongoing	In progress
Mulima	Support Adrian with frontend implementation, the design-system decision, and frontend reviews.	Sprint 1	Open
Adrian	Complete the wireframes, propose the common design system/template, and begin the React pages and reusable components.	Sprint 1	In progress

Owner	Action	Target	Status
Adrian	Keep frontend work on feature branches and raise pull requests for review.	Immediate / ongoing	In progress
Samar	Prepare and share the branch-naming and workflow guide.	By July 16, 2026	Open
Samar	Review the existing CI setup and configure or propose branch protection, required reviews, test gates, and merge controls.	Early Sprint 1	Open
Samar	Support backend and test-suite work, enable relevant tests, and progress DevOps/VPS work without displacing critical feature stories.	Sprint 1	Open
All members	Read the repository READMEs and architecture documents and raise corrections or additions through pull requests.	Immediate	Open
All members	Confirm access by cloning the repository, creating a branch, and testing the pull-request workflow.	Immediate	In progress
All members	Assign Trello cards before starting work, keep status current, follow priority order, and raise blockers promptly.	Ongoing	Open
Team	Write a short architecture/repository decision record covering the active repository, admins, branch rules, deployment source, domain, and treatment of the earlier repository.	Early Sprint 1	Open
Team	Review and approve the API contracts and confirm whether the MVP will be a modular monolith or separately deployed services.	Early Sprint 1	Open
Team	Continue weekly stand-ups/alignment meetings and use chat between meetings.	Weekly	Agreed

17. Risks and Open Questions

Risk / Issue	What raised it	Possible impact	Next step
Repository governance	Two repositories existed before the meeting.	Duplicate work, split history, or confusion over the active codebase.	Keep one active development repository and document administration and deployment responsibilities.
Schedule pressure	Sprint 0 ended with setup work still open.	Less time for integration, testing, documentation, and presentation preparation.	Protect the critical path, keep ownership visible, and integrate continuously.
Architecture wording	The diagrams use microservices language while the code may remain one deployable.	The final report could describe something different from what was actually built.	Confirm the target implementation and update the diagrams and ADR.
LLM reliability	External model output cannot be treated as verified fact.	Unsafe or misleading recommendations and weak academic defensibility.	Use schema validation, evidence, caveats, deterministic tests, and clear limitations.
Risk labels	The labels and colors were discussed but not finalized.	Backend, frontend, and tests may implement different meanings.	Ahmed to define one stable contract and user-facing interpretation.
Design consistency	Multiple people may build frontend pages independently.	A fragmented user experience and avoidable rework.	Adrian and Mulima to establish shared components and style guidance early.
Access and permissions	Several usernames or invitations were initially missing or mismatched.	Blocked branches, reviews, or CI setup.	Verify that every member can clone, branch, open PRs, review, and merge as intended.
Quality gates	The original boilerplate PR was merged without a reviewer.	Unreviewed architecture or defects could enter main.	Enable branch protection and required checks now that the team has access.
Trello accuracy	Card ownership and status can become stale.	Duplicate work or hidden inactivity.	Members update their cards; Mulima checks the board regularly.
Deployment ownership	Development moved to one repository while the domain and earlier deployment setup belong to Ahmed.	Confusion over CI/CD triggers and operational responsibility.	Document and test the deployment path early.
Team capacity	Five part-time members must cover product, frontend, backend, AI, QA, DevOps, and documentation.	Bottlenecks and uneven workload.	Use clear ownership, pairing, and additional cards only after primary commitments are under control.
Documentation drift	READMEs and diagrams may fall behind the implementation.	The final report may not match the delivered system.	Update documentation in the same pull requests as the relevant code changes.

18. Sprint 1 Priorities

Sprint 1 should produce a working, integrated foundation rather than a collection of disconnected setup tasks. The team's working priorities are:

1. Confirm repository access, branch protection, naming conventions, reviewer expectations, and CI checks.

2. Review and approve the API contracts for authentication, listings, and analysis.
3. Complete the common design system and core wireframes.
4. Implement the authentication and user-identity foundation.
5. Implement listing submission, validation, storage, and history foundations.
6. Implement the deterministic/mock analysis path and provider interface before depending on live LLM calls.
7. Finalize the categorical risk labels and the analysis results contract.
8. Connect an initial React flow to working backend endpoints.
9. Keep Docker and deployment work moving in parallel and preserve the working health check.
10. Enable and pass the story tests and keep Trello and documentation current.
11. Finish the sprint with something that can be demonstrated end to end, even if the analysis is still using a deterministic mock.

19. Next Meeting

The team will continue meeting weekly. The next mid-sprint meeting will be treated as a stand-up or alignment session. The exact date and time will be confirmed through the team's normal communication channel.

Suggested agenda for the next meeting:

- Confirm that everyone can clone, branch, open pull requests, review, and merge according to their permissions.
- Review the repository and deployment decision record.
- Check Sprint 1 ownership and identify any blocked or unassigned critical cards.
- Review the design system and any updated architecture diagrams.
- Review progress on authentication, listings, analysis/provider work, and tests.
- Confirm the final risk labels and API response contract.
- Review branch protection, CI status, and initial deployment progress.
- Demonstrate the latest integrated build and update the risk list.

20. Closing Summary

The meeting moved the project from setup into active development. The team was broadly comfortable with the proposed architecture and boundaries, confirmed React and FastAPI, and agreed that shared contracts were necessary for parallel work. The group also decided that TrustAI should present explainable risk categories rather than unsupported probability percentages.

Responsibilities were aligned with each person's strengths, the task-branch and pull-request workflow was agreed, and the team chose the runnable boilerplate repository as the active development repository. Access was resolved during the call, and Sprint 1 was set to begin immediately. The main priorities now are to keep ownership visible, review the contracts and documentation carefully, integrate early, and raise blockers between meetings rather than waiting for the next call.